VR Library - Class Diagrams

Daoqi Xiong
ID: 25335365

Jiayun Zhang
ID: 25334655

Joshua O'Donnell
ID: 25353241

Keshav Tripathi
ID: 25347551

Priyansh Nayak
ID: 25350660

Rundong Chen
ID: 25336554

Viraj Bhor
ID: 25358463

Xu Gong
ID: 25334562

Yokesh Kathiravan
ID: 25341409

Group 3

1 Subsystem Diagrams

The following class diagrams present the main subsystems of the VR Library project. Rather than modelling every Unity script individually, the diagrams focus on the structural backbone of the system by showing the principal managers, services, controllers, and networking components. Together, they describe how authentication, room persistence, multiplayer session control, book interaction, and shared world features are organised across the application.

1.1 Authentication and Backend Access

This diagram models the authentication and backend communication layer. It shows how `LoginUI` interacts with `APIManager`, which acts as a facade over the domain services responsible for authentication, users, roles, rooms, and books. It also includes `RequestUtils` for HTTP-based communication and `WebSocketManager` for persistent realtime server messaging after login.

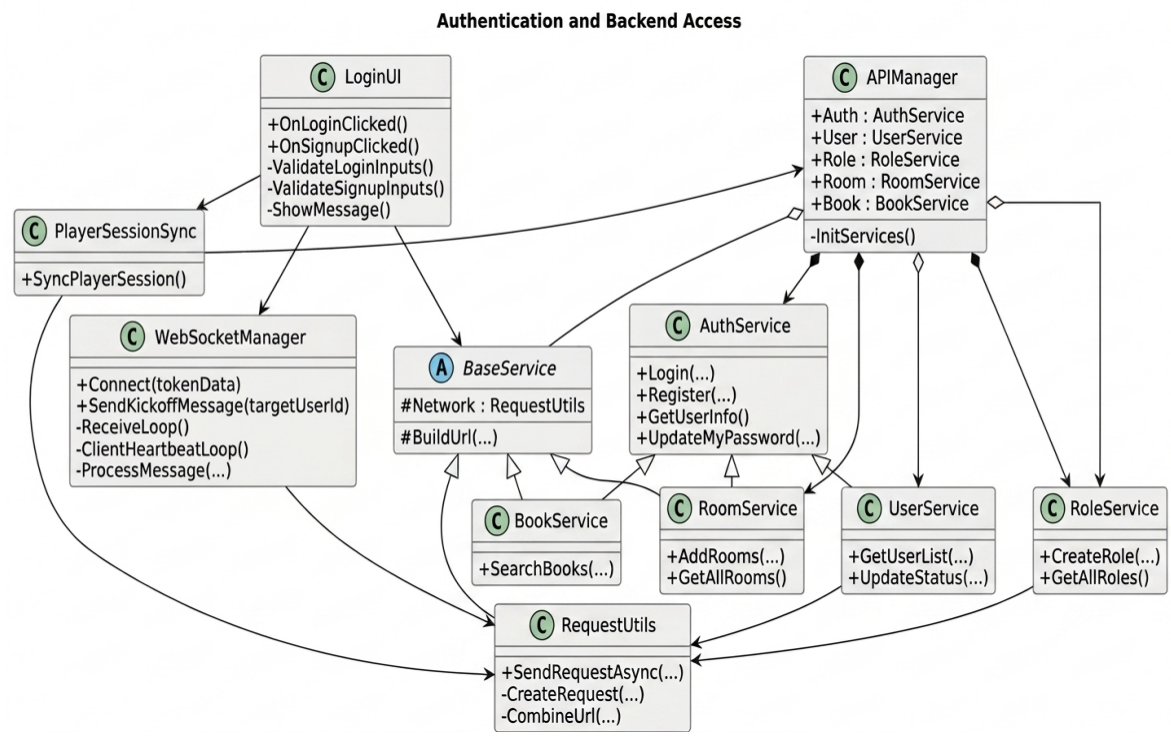

Figure 1: Authentication and Backend Access Architecture

1.2 Room Persistence and Cache

This diagram describes the room persistence subsystem. The central class, `RoomCacheService`, manages the in-memory room collection and applies lifecycle rules such as upserting, status updates, dormancy, and expiry cleanup. Persistence is abstracted through the `IRoomStore` interface, allowing the system to switch between local file-backed caching and server-backed room storage.

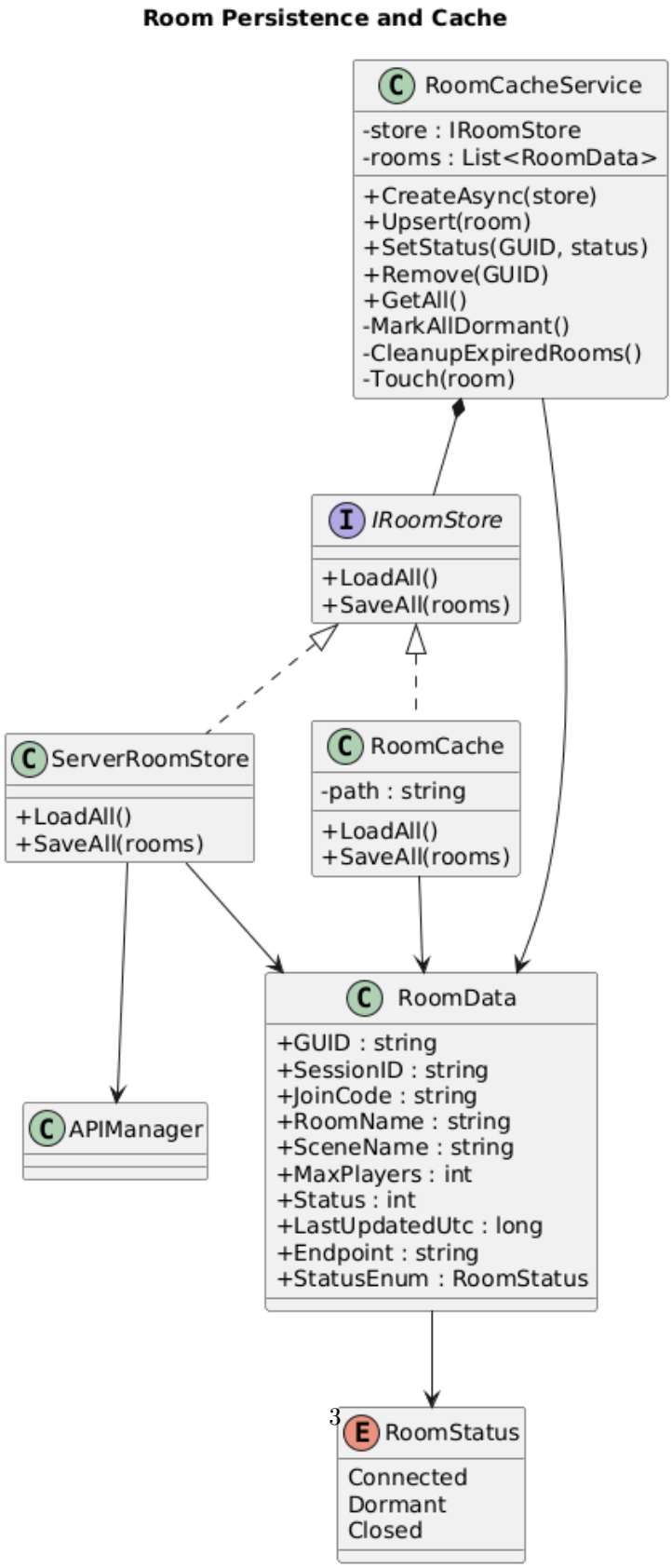

Figure 2: Room Persistence and Cache Architecture

1.3 Multiplayer Session Core

This diagram models the high-level multiplayer control layer. `XRINetworkGameManager` coordinates connection state, player state, and session flow while delegating responsibilities to `SessionManager`, `AuthenticationManager`, `VoiceChatManager`, and the underlying multiplayer network manager. It therefore represents the core structure responsible for establishing and maintaining networked VR sessions.

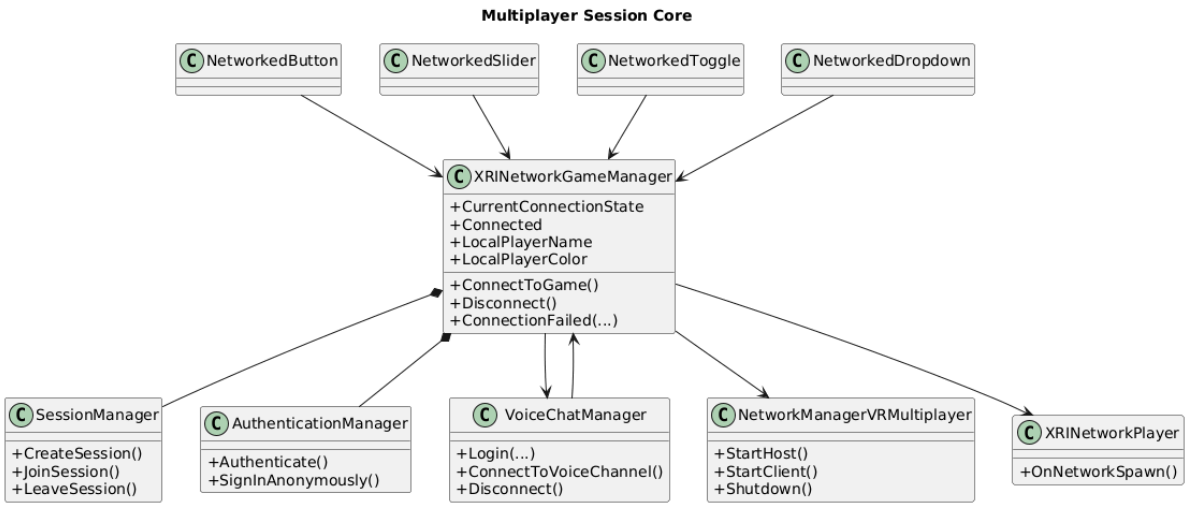

Figure 3: Multiplayer Session Core Architecture

1.4 Book System

This diagram captures the main custom feature of the application: the interactive book subsystem. `BookSystem` acts as the central orchestrator, receiving book requests, resolving book data, spawning local or networked books, and connecting them to controllers, renderers, pagination logic, and interaction handlers. The networking side is represented through `NetworkBookManager`, `NetworkBookState`, and `NetworkBookSync`, which together manage distributed shared book state.

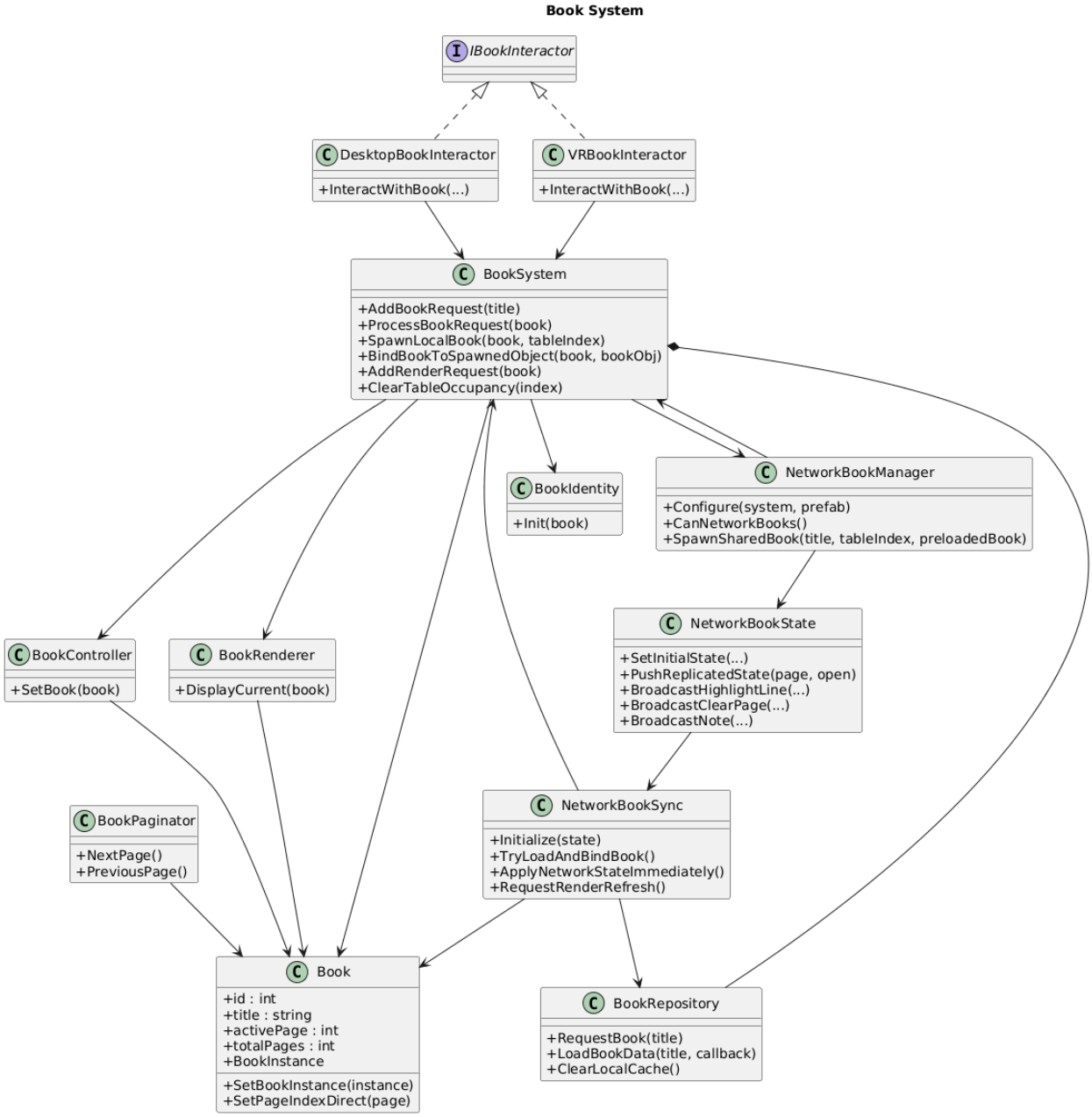

Figure 4: Book System Architecture

1.5 Interactive World Systems

This diagram models the shared interactive environment systems outside the book feature. It includes the networked blackboard classes, which handle drawing, board tools, interaction zones, and stroke replication, alongside the day-night and weather controllers, which modify environmental conditions over time. These classes together represent the world-level interactive features of the VR library.

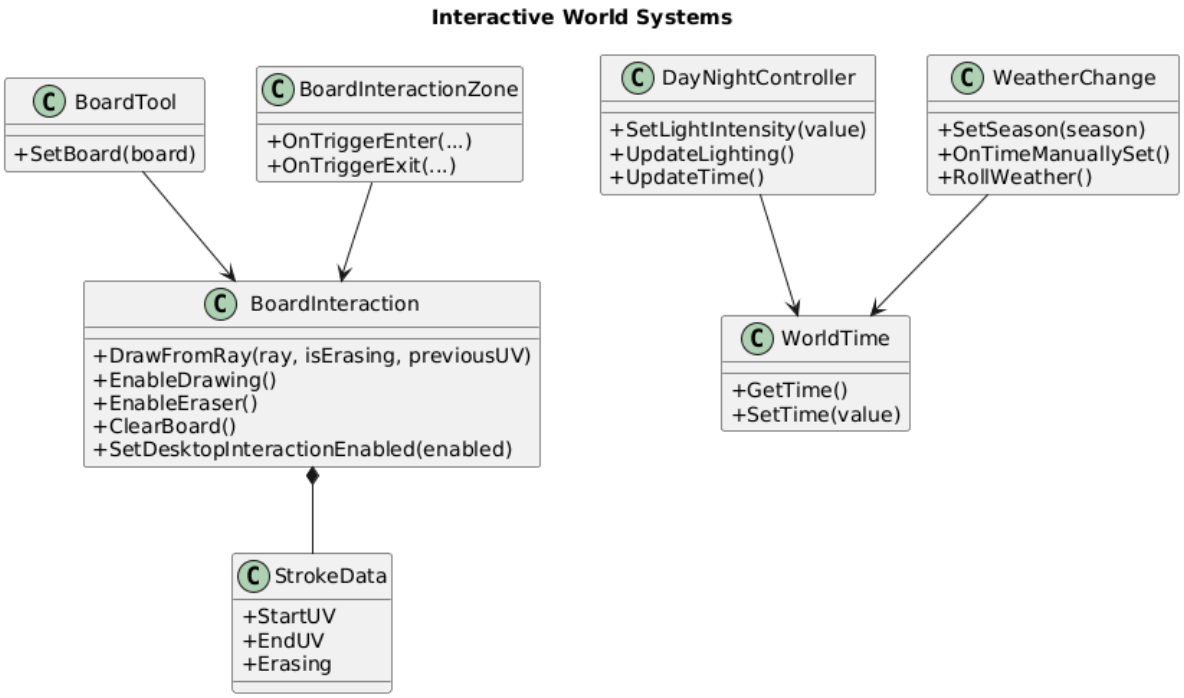

Figure 5: Interactive World Systems

1.6 Book Discovery and Request UI

This diagram describes the flow between book discovery and the main book subsystem. `FetchBooks` handles search and result population, `BookRowUI` represents the request entry point from the user interface, and `BookSystem` and `BookRepository` take over once a book is requested. The diagram therefore shows how external search results are transformed into in-world book interactions.

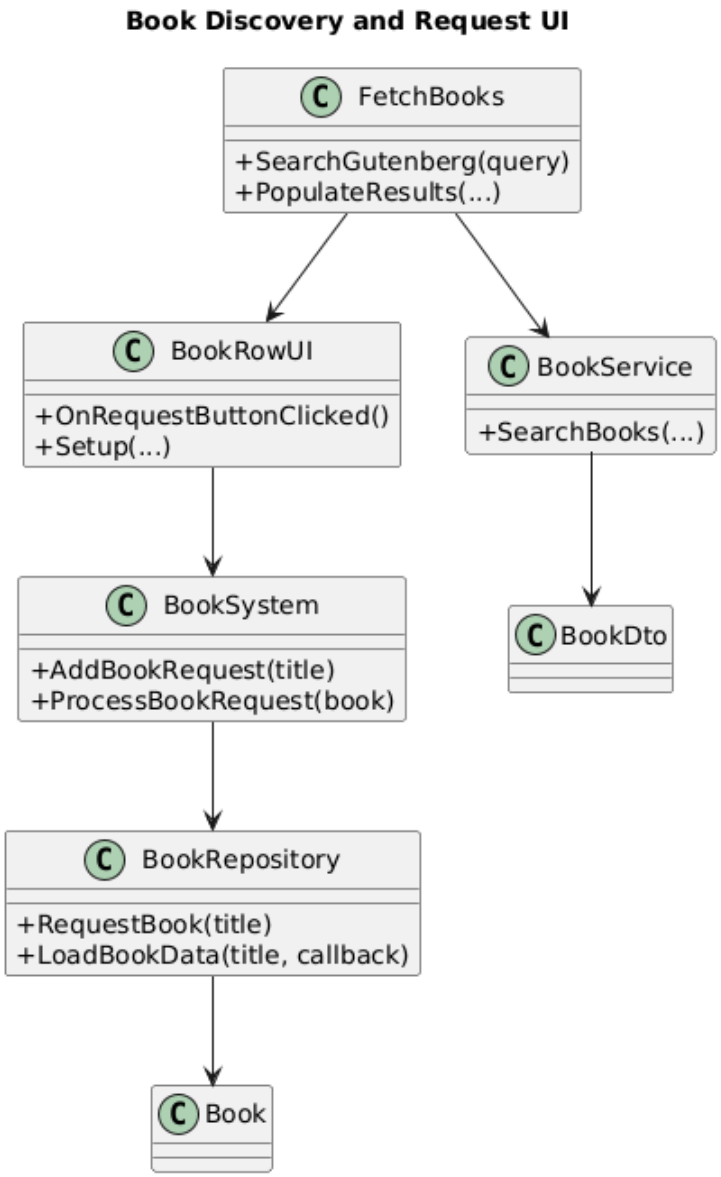

Figure 6: Book Discovery and Request UI

1.7 Player and Interaction Networking

This final diagram focuses on the Unity-side player and interactable object structure in the shared VR space. It models the networked player entity, including avatar IK, visuals, and player name display, together with the main networked

interactable classes used for shared object manipulation and synchronization. This subsystem is placed last because it is the most implementation-oriented and most closely tied to Unity's multiplayer interaction layer.

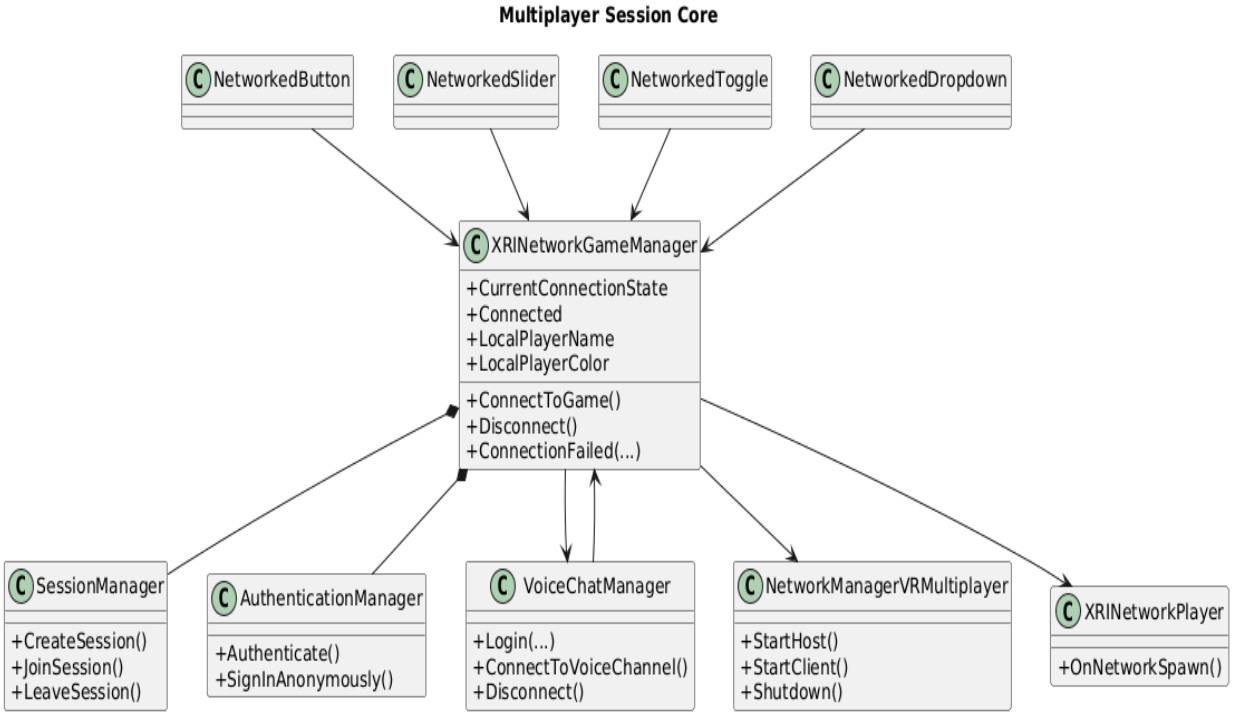

Figure 7: Player and Interaction Networking